

CONTROLLED SOFTWARE DESIGN SPECIFICATION

Core Software Design Requirements Specification

CSDRS controlled master document

Current controlled issue

Data Identity, Audit History and Temporal Versioning

Includes the Stable-Identity Temporal Audit Chain (SITAC) and supporting database identity requirements.

DOCUMENT REFERENCE

AUTHOR AND OWNER

PUBLISHER

VERSION AND ISSUE DATE

CSDRS

Paul Emerton

SPAWN.LONDON

Version 1.0 | 18 August 2026

Document Index

Document Control	3
Document Governance and Maintenance.....	4
Data Identity, Audit History and Temporal Versioning	5
CSDRS-DATA-001 — Stable-Identity Temporal Audit Chain (SITAC)	6
Core Principle.....	6
Required Audit Fields	6
Creating a Record.....	7
Editing a Record.....	8
Example	8
Meaning of Audit Timestamps	9
Soft Deletion.....	10
Restoration	11
Transaction and Concurrency Requirement.....	12
Historical Data Is Immutable	13
Archive and Retention.....	14
Independent Integrity / Compromise Recovery	15
CSDRS-DATA-002 — UUID-Only Persistent Identity Principle	16
CSDRS-DATA-003 — Audit Metadata Trust Boundary	17
CSDRS-DATA-004 — Security Logging Is Separate From SITAC	18
AI IMPLEMENTATION INSTRUCTION	19

The index is generated from controlled Heading 1, Heading 2 and Heading 3 styles. It must be refreshed whenever the specification is amended.

Document Control

Controlled-document status. ISO/IEC 27001 does not prescribe a visual template. This format is designed to support the control of documented information through clear identification, ownership, approval, versioning, access, review, retention and disposition. Conformance depends on operation within the organisation’s Information Security Management System.

Document identification

Document title	Core Software Design Requirements Specification
Document reference	CSDRS
Document owner	Paul Emerton
Author	Paul Emerton
Publisher	SPAWN.LONDON
Information classification	Public
Document status	Published controlled issue
Version	1.0
Effective date	18 August 2026
Reference standard	ISO/IEC 27001:2022 and applicable amendments

Review and approval

Approved by	Paul Emerton
Approval date	18 August 2026
Review frequency	At least annually and whenever a material architectural, legal, regulatory or security change occurs
Next scheduled review	18 August 2027

Revision history

Version	Issue date	Status	Change summary	Approved by
1.0	18 August 2026	Published	Initial controlled issue. Includes CSDRS-DATA-001 to CSDRS-DATA-004 and the AI implementation instruction.	Paul Emerton

Document Governance and Maintenance

Document hierarchy and expansion convention

This master specification is designed to expand beyond the current SITAC requirements. **Heading 1** identifies a requirement family, **Heading 2** identifies an individual requirement or directive, and **Heading 3** identifies implementation detail. New requirements should use the stable form **CSDRS-[DOMAIN]-[NNN]**. Published requirement identifiers must not be renumbered merely because further sections are inserted.

Controlled copy notice

- The approved electronic master is the controlled copy.
- Printed, exported or locally downloaded copies are uncontrolled unless separately registered and marked.
- Changes require documented review, approval, a version increment and an entry in the revision history.
- Superseded issues must be withdrawn from active use and retained or disposed of in accordance with the applicable retention schedule.
- Access, distribution and storage must follow the information classification and the organisation's ISMS procedures.

Normative language

MUST / MUST NOT mandatory requirement; **SHOULD / SHOULD NOT** recommended requirement unless a documented exception is approved; **MAY** permitted option.

Amendment and publication workflow

1. Assess the proposed change for architectural, security, legal and operational impact.
2. Retain all published requirement identifiers and allocate new identifiers sequentially within the relevant domain.
3. Complete technical review and obtain approval from the document owner or authorised approval authority.
4. Update the version, effective date, revision history, document index and any affected cross-references.
5. Publish the approved master, withdraw the superseded issue from active use, and archive it under the applicable retention rules.

Index maintenance

The hierarchical index on page 2 is generated from Heading 1 to Heading 3. **All document fields must be refreshed before an amended issue is approved or published.**

Data Identity, Audit History and Temporal Versioning

SPECIFICATION FAMILY | CSDRS-DATA

The following requirements are **mandatory architectural requirements**. When designing, generating, modifying, reviewing or refactoring software, you **MUST** preserve these principles unless explicitly instructed otherwise.

CSDRS-DATA-001 — Stable-Identity Temporal Audit Chain (SITAC)

MANDATORY ARCHITECTURAL REQUIREMENT

This application uses **SITAC — Stable-Identity Temporal Audit Chain**, our application-level implementation of temporal data versioning.

Core Principle

CORE PRINCIPLE
Every logical database entity has a permanent canonical UUID.

The canonical/live database row **MUST** retain this UUID throughout its lifecycle.

Editing a record **MUST NOT** replace the canonical record with a new record and **MUST NOT** require foreign keys elsewhere in the database to be changed.

Instead, SITAC uses **copy-before-write versioning**.

Required Audit Fields

Auditable database records **MUST** include:

id	UUID PRIMARY KEY
created_by	UUID
created_at	UTC timestamp
deleted_by	UUID NULL
deleted_at	UTC timestamp NULL
originating_id	UUID NULL

These fields are controlled by the server/application persistence layer. They **MUST NOT** be trusted from client-supplied JSON or form data.

Creating a Record

When a new record is created:

```
id          = new permanent canonical UUID
created_by   = authenticated user UUID
created_at   = current trusted UTC timestamp
deleted_by   = NULL
deleted_at   = NULL
originating_id = NULL
```

The id becomes the permanent identity of that logical entity.

Editing a Record

An edit **MUST** be performed atomically inside a database transaction.

Before modifying the canonical row:

1. Lock/read the current canonical row.
2. Generate a new UUID for a historical snapshot.
3. Copy the complete current canonical row into the snapshot.
4. Give the snapshot the new UUID.
5. Preserve the old created_at.
6. Preserve the old created_by.
7. Copy the canonical row's existing originating_id into the snapshot.
8. Set the snapshot's deleted_at to the current edit timestamp.
9. Set the snapshot's deleted_by to the authenticated editing user.
10. Insert the historical snapshot.

Then modify the existing canonical row:

```
id           = UNCHANGED
business data = updated values
created_at   = edit timestamp
created_by   = editing user UUID
deleted_at   = NULL
deleted_by   = NULL
originating_id = UUID of snapshot just created
```

The canonical id **MUST NEVER** change during an edit.

Existing foreign keys therefore continue to reference the same UUID and require no modification.

Example

Initial creation:

```
A
data           = Version 1
created_at     = T0
created_by     = User 1
originating_id = NULL
```

After first edit:

```
CANONICAL RECORD

A
data           = Version 2
created_at     = T1
created_by     = User 2
originating_id = H1
deleted_at     = NULL
```

Historical snapshot:

```
H1
data           = Version 1
created_at     = T0
created_by     = User 1
originating_id = NULL
deleted_at     = T1
deleted_by     = User 2
```

After another edit:

```
A -> H2 -> H1 -> NULL
```

Where:

- A is always the permanent canonical UUID.
- H2 contains the state immediately before the latest edit.
- H1 contains the state before that.
- `originating_id` is therefore a backward pointer to the immediately preceding version.

After any number of edits:

CURRENT CANONICAL RECORD

```
  |
  v
previous version
  |
  v
previous version
  |
  v
...
  |
  v
original version
```

The complete history of the entity can therefore be reconstructed without changing the UUID referenced by the rest of the application.

Meaning of Audit Timestamps

Within SITAC:

`created_at` means the time at which **that version became effective**.

`created_by` identifies the actor who created **that version**.

For a historical snapshot:

`deleted_at` means the time at which that version **ceased to be the current version**.

`deleted_by` identifies the actor whose action superseded that version.

For the canonical record:

`deleted_at` and `deleted_by` indicate that the logical entity itself has been **soft deleted**.

Soft Deletion

Normal application deletion **MUST NOT** physically delete a SITAC canonical record.

To delete an entity:

```
deleted_at = trusted UTC timestamp  
deleted_by = authenticated user UUID
```

The canonical:

```
id  
business data  
created_at  
created_by  
originating_id
```

remain intact.

Normal application queries **MUST** exclude soft-deleted records unless deleted-record access is explicitly required.

Restoration

Restoration MUST itself be auditable.

Before restoring a deleted canonical record:

1. Copy its deleted state into a new historical snapshot.
2. Preserve the deletion information in that snapshot.
3. Link the snapshot into the existing `originating_id` chain.
4. Clear `deleted_at` and `deleted_by` from the canonical record.
5. Set the canonical `created_at` to the restoration timestamp.
6. Set `created_by` to the restoring user.
7. Point the canonical `originating_id` to the newly created snapshot.

The canonical UUID MUST remain unchanged.

This preserves evidence that the entity was deleted and subsequently restored.

Transaction and Concurrency Requirement

Creation of a historical snapshot and modification of its canonical row **MUST** form **one atomic database transaction**.

The implementation **MUST** prevent concurrent edits from:

- overwriting one another;
- creating competing history branches;
- creating orphaned historical snapshots; or
- producing an incomplete history chain.

Use appropriate row locking and/or optimistic concurrency protection.

Either the complete SITAC operation succeeds or the complete operation rolls back.

Historical Data Is Immutable

Once a historical snapshot has been created, normal application code **MUST NOT** modify it.

Historical snapshots **MUST NOT** be physically deleted by ordinary CRUD operations.

Administrative interfaces **MUST** distinguish clearly between:

- current records;
- soft-deleted canonical records; and
- historical SITAC snapshots.

Never treat every record containing `deleted_at` as simply a deleted business record. Historical versions also contain `deleted_at`.

Archive and Retention

Historical SITAC snapshots MAY eventually be moved from operational tables into:

- archive tables;
- immutable storage;
- cold storage;
- database backup archives; or
- structured JSON archives.

Archival MUST preserve:

id
canonical_id
created_by
created_at
deleted_by
deleted_at
originating_id
business data
entity/table type
schema version
archive timestamp

A historical UUID MUST remain resolvable for as long as the applicable retention policy requires that history to exist.

Historical data MUST NOT be retained indefinitely merely because SITAC makes retention possible. Applicable legal, contractual, UK GDPR and organisational retention requirements still apply.

Independent Integrity / Compromise Recovery

The SITAC chain provides **audit history and structural integrity**, but it **MUST NOT** be represented as cryptographically tamper-proof.

An attacker with sufficient database privileges could potentially alter historical records and their links.

CRC values **MUST NOT** be relied upon as a security mechanism.

Cryptographic hashes **MAY** be used as an additional integrity mechanism, but hashes stored solely alongside the database they protect do not independently prove that a privileged attacker has not rewritten the history and recalculated the hashes.

The preferred high-level protection is:

SITAC operational history
+
independently secured backups/archives
+
separate security boundary
+
documented retention

Where required by the risk assessment, periodic database/archive manifests **MAY** additionally contain SHA-256 or stronger cryptographic hashes and be stored with independently protected backups.

Cold/immutable backups **SHOULD** use credentials and infrastructure that prevent a compromise of the production application from silently rewriting previous backup generations.

CSDRS-DATA-002 — UUID-Only Persistent Identity Principle

MANDATORY ARCHITECTURAL REQUIREMENT

Persistent domain entities **MUST** use UUIDs as their authoritative identities.

Do **NOT** use sequential integer auto-increment IDs as authoritative domain identities.

Preferred structure:

id UUID PRIMARY KEY

Foreign keys **MUST** reference the appropriate UUID:

customer_id UUID
project_id UUID
user_id UUID
organisation_id UUID

A UUID:

- **MUST** be created once;
- **MUST** remain immutable;
- **MUST** never be reused;
- **MUST** survive editing;
- **MUST** survive soft deletion;
- **MUST** survive restoration; and
- **MUST** remain associated with archived historical data.

UUIDv7 **SHOULD** normally be used for new systems where supported.

UUIDv4 **MAY** be used where random identifiers are preferable or required.

Sequential human-readable numbers **MAY** still be used for business purposes such as:

invoice_number
order_number
case_number
display_reference

but these are business attributes and **MUST NOT** replace the entity UUID.

A UUID is an identifier, **not an authorisation mechanism**. Every API operation **MUST** still perform appropriate authentication and object-level authorisation.

CSDRS-DATA-003 — Audit Metadata Trust Boundary

MANDATORY ARCHITECTURAL REQUIREMENT

Audit metadata **MUST** be generated or established by trusted server-side components.

Client requests **MUST NOT** be permitted to arbitrarily establish or alter:

```
id
created_by
created_at
deleted_by
deleted_at
originating_id
```

The authenticated server session determines actor identity.

The trusted application/database clock determines audit timestamps.

All persistent audit timestamps **MUST** be stored in UTC.

Application presentation layers **MAY** convert UTC timestamps into the user’s local timezone for display.

CSDRS-DATA-004 — Security Logging Is Separate From SITAC

MANDATORY ARCHITECTURAL REQUIREMENT

SITAC records **changes to application data**.

It is NOT a substitute for application and security event logging.

The application **MUST** separately log security-relevant events where appropriate, including:

- authentication success/failure
- authorisation failure
- administrative actions
- permission changes
- data exports
- security configuration changes
- exceptional data access
- system/service activity
- backup/archive operations
- purge operations

Where useful, related changes **SHOULD** share an operation_id, request_id or equivalent correlation UUID so that a business-data change can be correlated with the corresponding security/application event.

AI IMPLEMENTATION INSTRUCTION

MANDATORY AI IMPLEMENTATION DIRECTIVE

When generating or modifying code for this application:

DO NOT replace SITAC with conventional CRUD, hard deletes, event sourcing, database-specific temporal tables, or another audit-history design unless explicitly instructed to change the architecture.

If existing code does not comply with these requirements:

1. Identify the conflict.
2. Explain the consequence.
3. Propose a SITAC-compliant implementation.
4. Preserve existing data and relationships during migration.
5. Do not silently change the established architecture.

When designing new database entities, APIs, repositories, services, migrations or administrative functions, assume these requirements apply unless explicitly stated otherwise.

The overriding design objective is:

Maintain one permanent canonical UUID for each logical entity while retaining a complete, attributable and recoverable temporal history of every material change to that entity.

This architecture is called:

SITAC — Stable-Identity Temporal Audit Chain

Core concept: temporal data history without sacrificing stable identity.